Classes

- Objects of the same *type* share defintions of methods but carry their own data
- Each *type* of object is defined by a *class*
- You can make your own of *type* of object by defining a *class*

Værdier er “objekter”

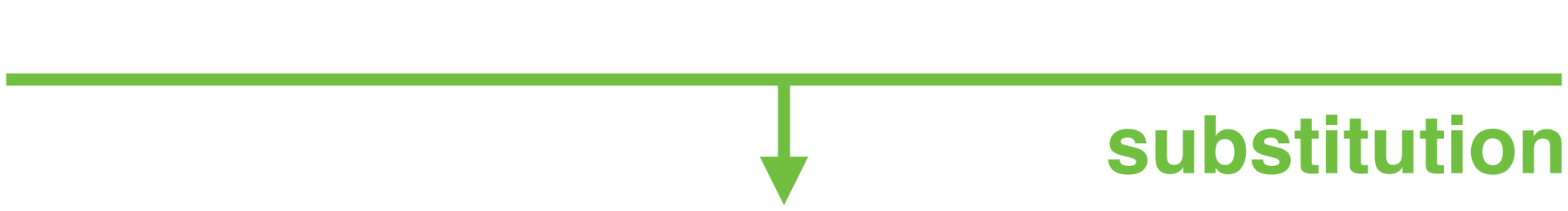
```
"banana"
```

Metoder

```
"banana".upper()
```

Metoder

```
"banana".upper()
```




```
"BANANA"
```

Types of objects

```
int str float bool
tuple list dict
```

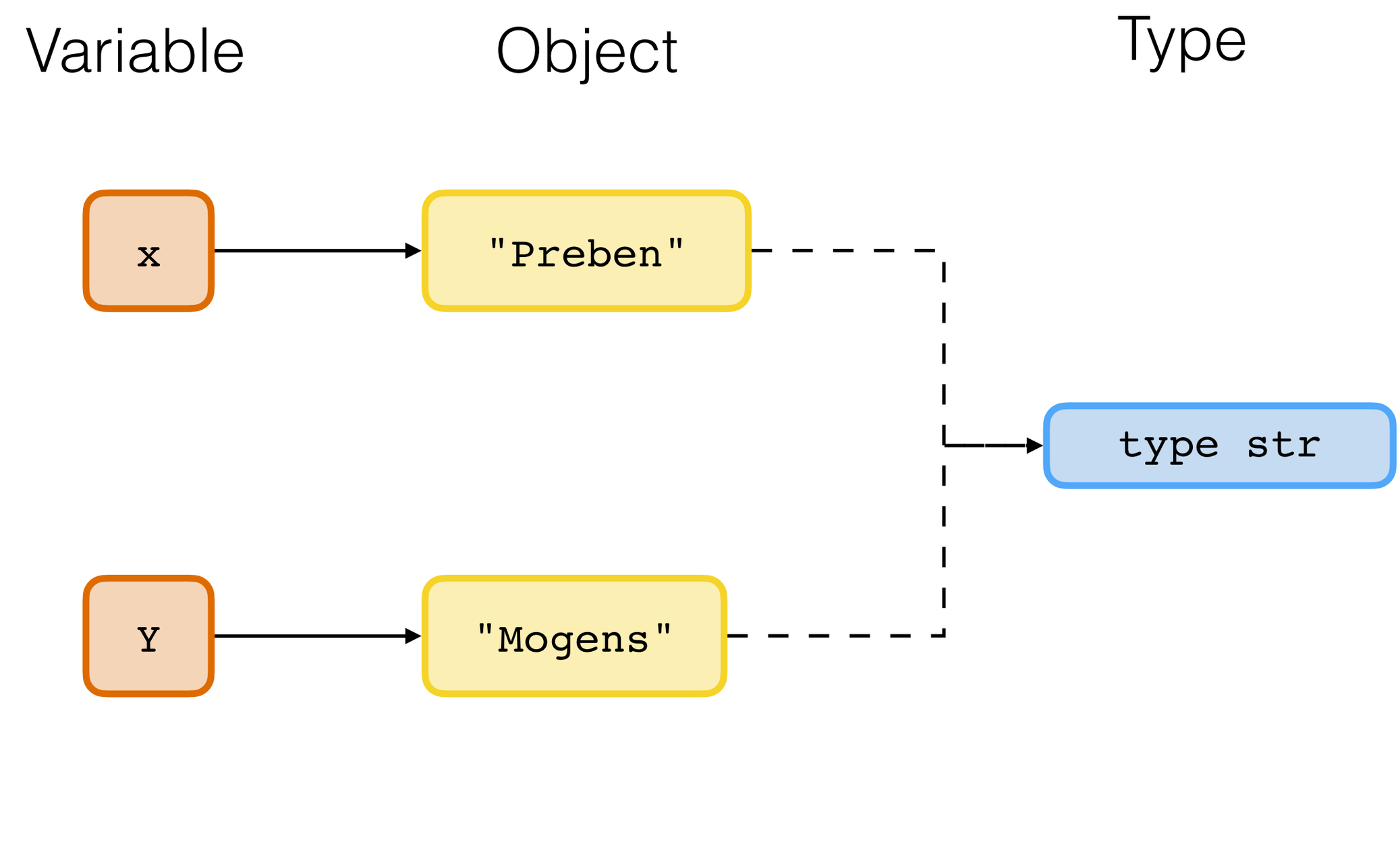
What type is my object?

```
fruit = "banana"
print( type(fruit) )
```

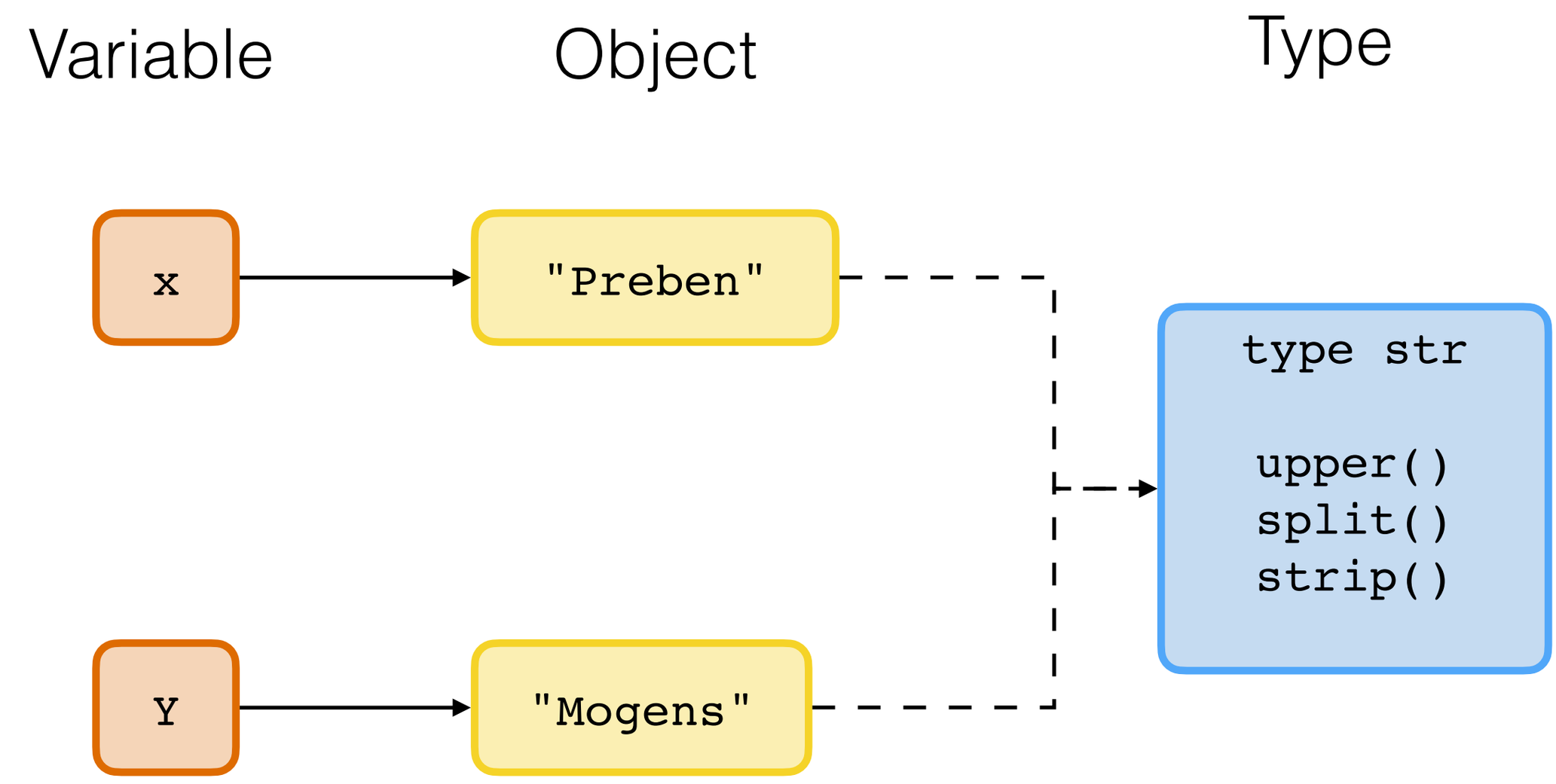
Prints:

```
str
```

All objects have a type



The type determines what an object can do



Calling methods

```
"banana".upper()
```

```
str.upper("banana")
```

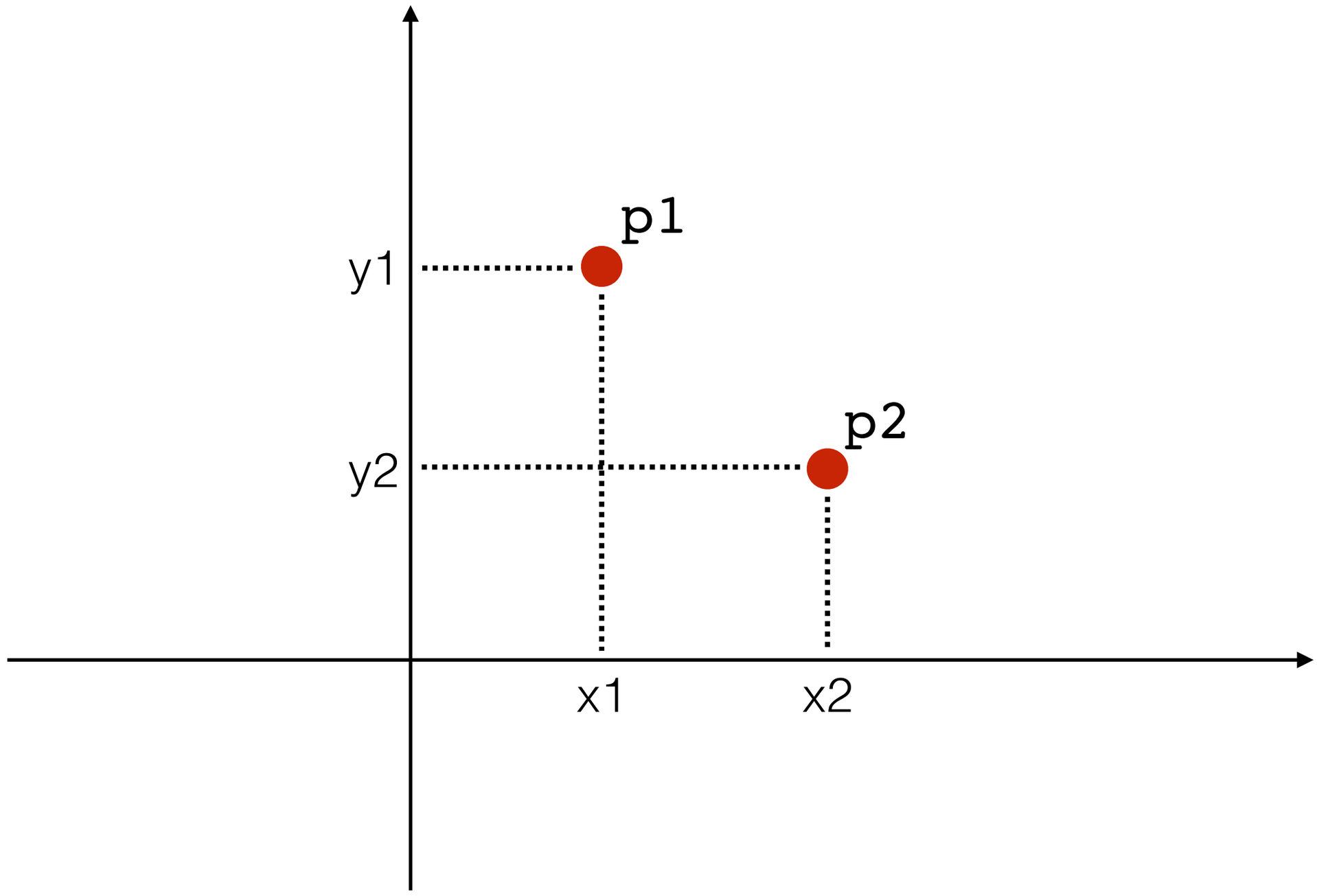
Classes define types

- **Types** and the functionality they provide are defined by **classes**.

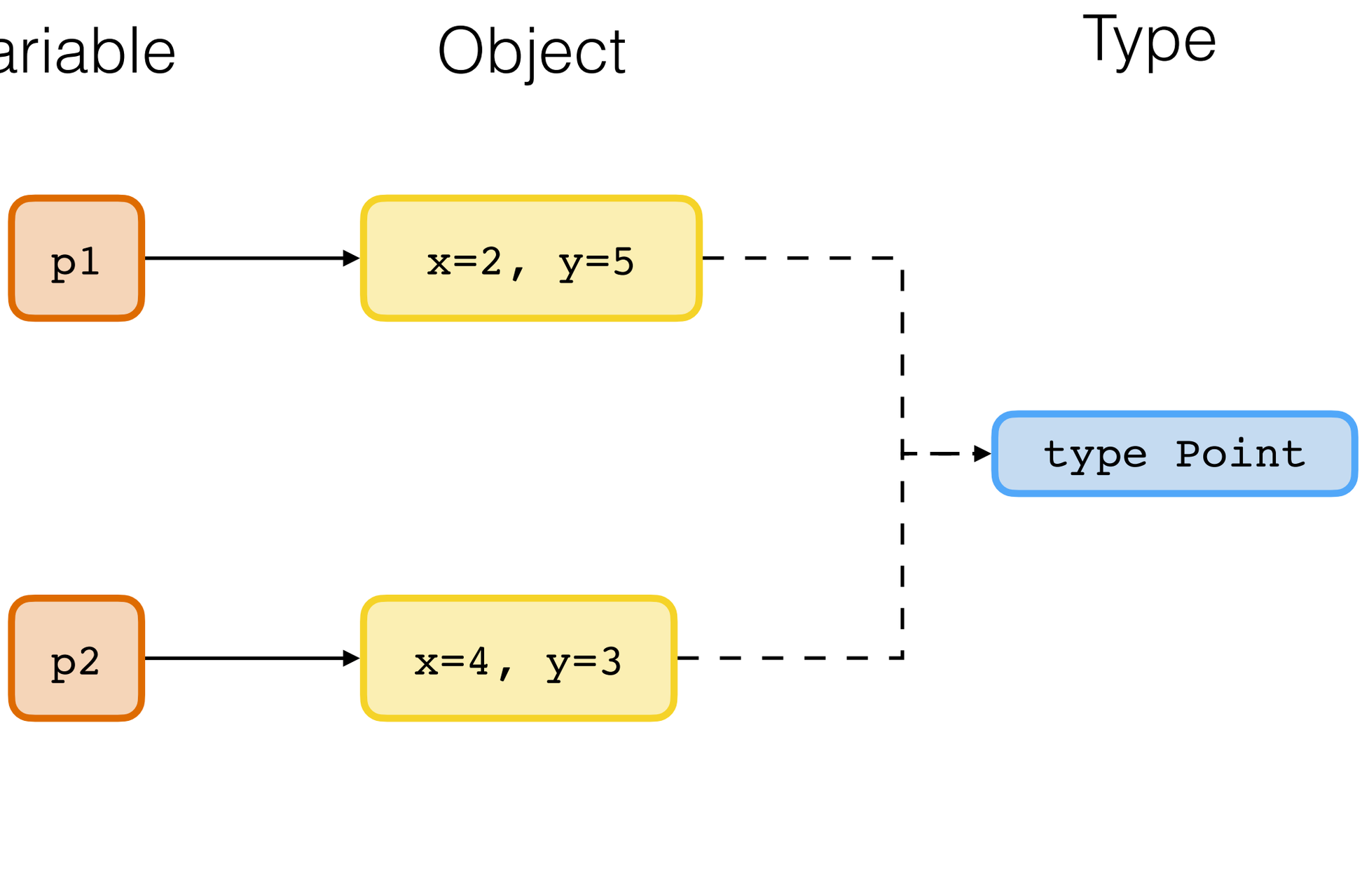
- By creating a **class** you define a new **type**

- The methods defined in a **class** is available to objects of the **type** it defines.

Points



A point class



`class` definition

```
class Point(object):
    ...

p = Point()
```

`object` is the generic object all objects start out as. For now, the objects our **`Point`** class make are not different from that.

`__init__` initializes

```
class Point(object):
    def __init__(self):
        ...

p = Point()
```

- `self` points to the generic **`object`** that is initialized.

`__init__` initializes

```
class Point(object):
    def __init__(self):
        ...

p = Point() # Point.__init__(object)
```

- `self` points to the generic **`object`** that is initialized.

`__init__` initializes

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

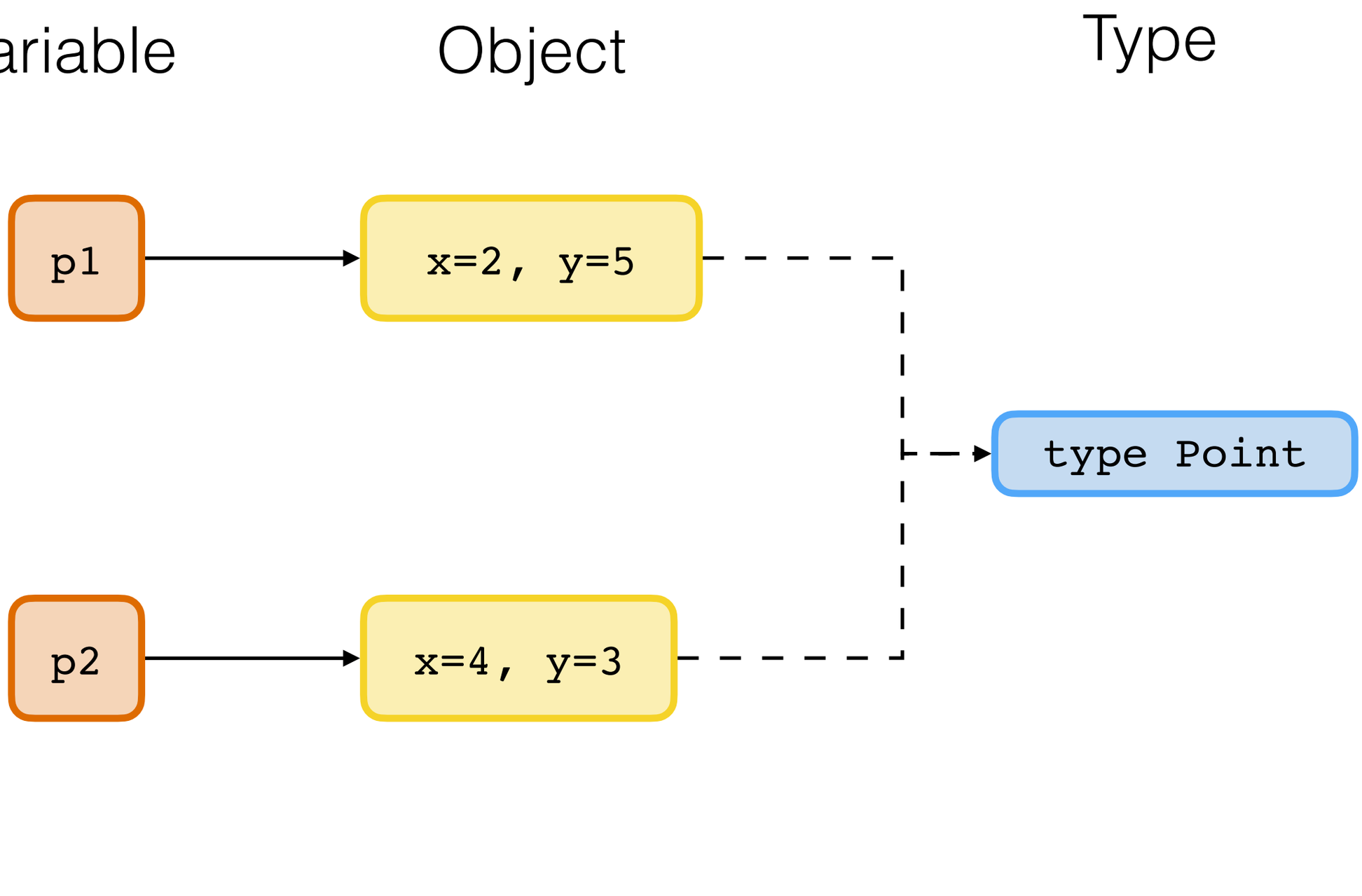
p1 = Point(2, 5)
p2 = Point(4, 3)
```

`__init__` initializes

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p1 = Point(2, 5) # Point.__init__(object, 2, 5)
p2 = Point(4, 3) # Point.__init__(object, 4, 3)
```

A `Point` class



A `Point` class

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
print(Point)
```

Prints:

```
<class '__main__.Point'>
```

An object of type `Point`

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p = Point(4, 3)

print(type(p))
```

Prints:

```
<class '__main__.Point'>
```

An object of type Point

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p = Point(4, 3)

print(type(p))
print(Point)
```

Prints:

```
<class '__main__.Point'>
<class '__main__.Point'>
```

An object of type `Point`

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p = Point(4, 3)
print(p)
```

Prints:

```
<__main__.Point object at 0x1032b4a20>
```

Access attributes

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p = Point(4, 3)

print(p.x)
print(p.y)
```

Prints:

```
4
3
```

How to look as a string

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p = Point(4, 3)
print(p)
```

Prints:

```
<__main__.Point object at 0x1032b4a20>
```

How to look as a string

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return ...

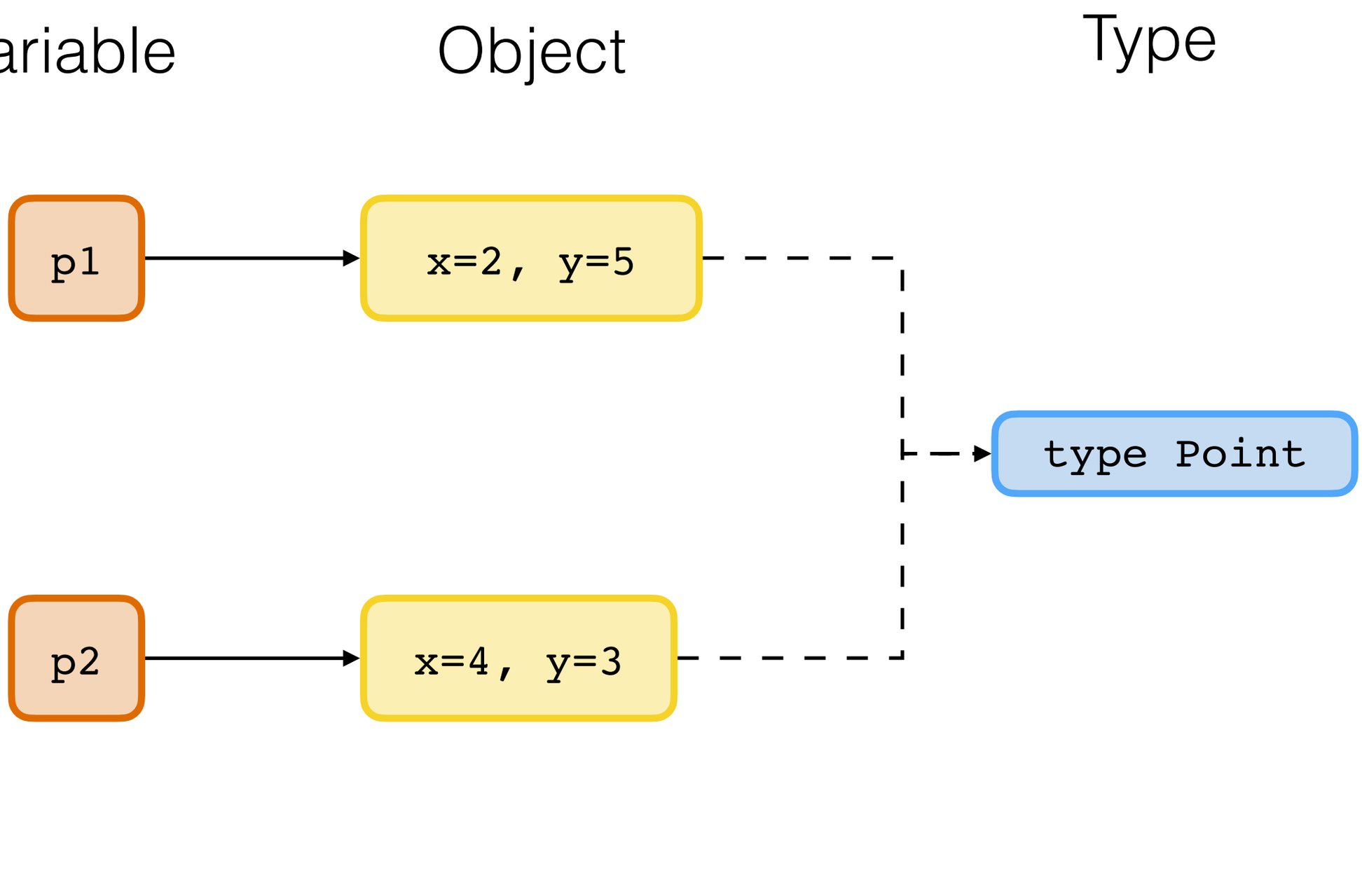

p = Point(4, 3)
print(p)
```

Prints:

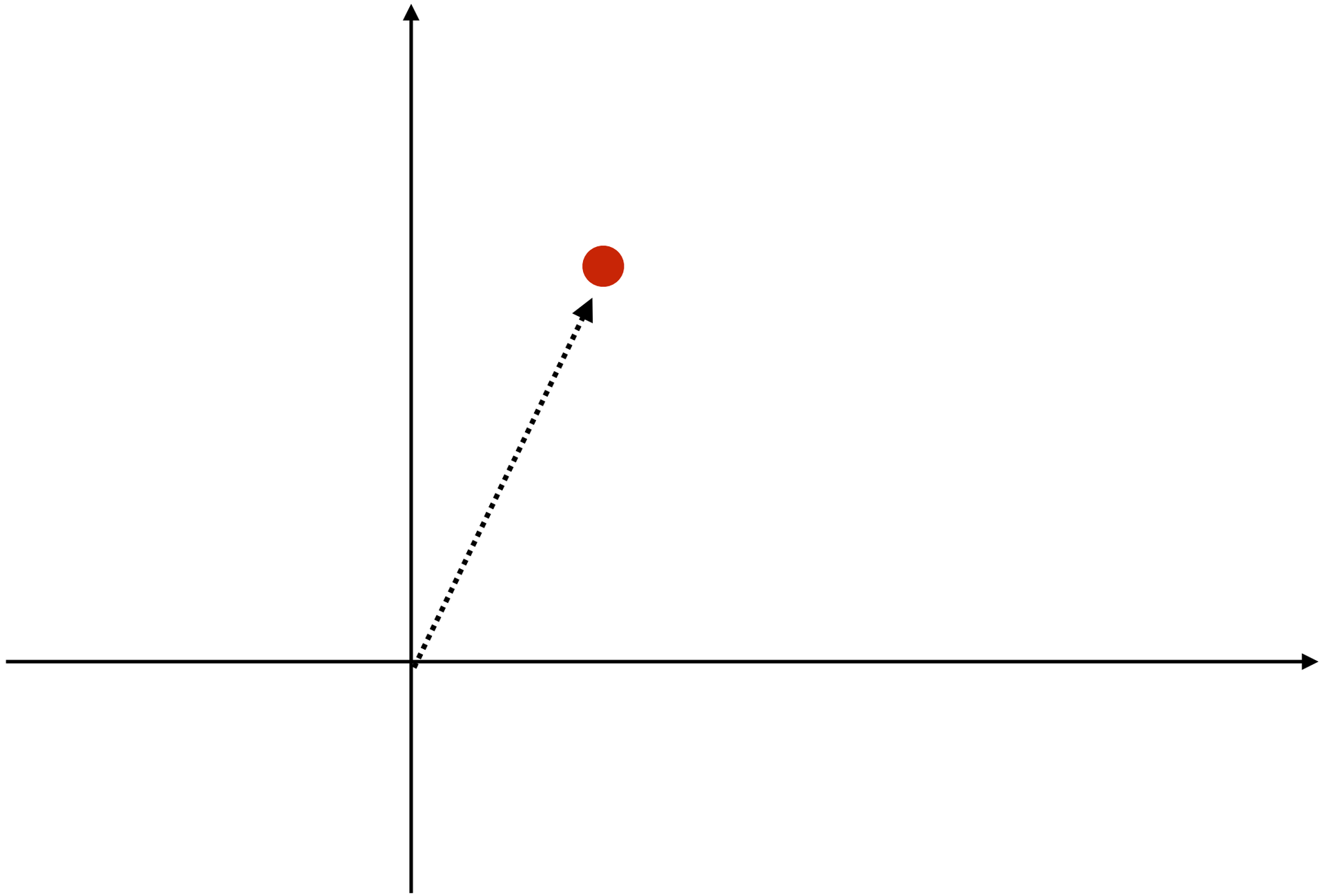
```
(4, 3)
```

Solution

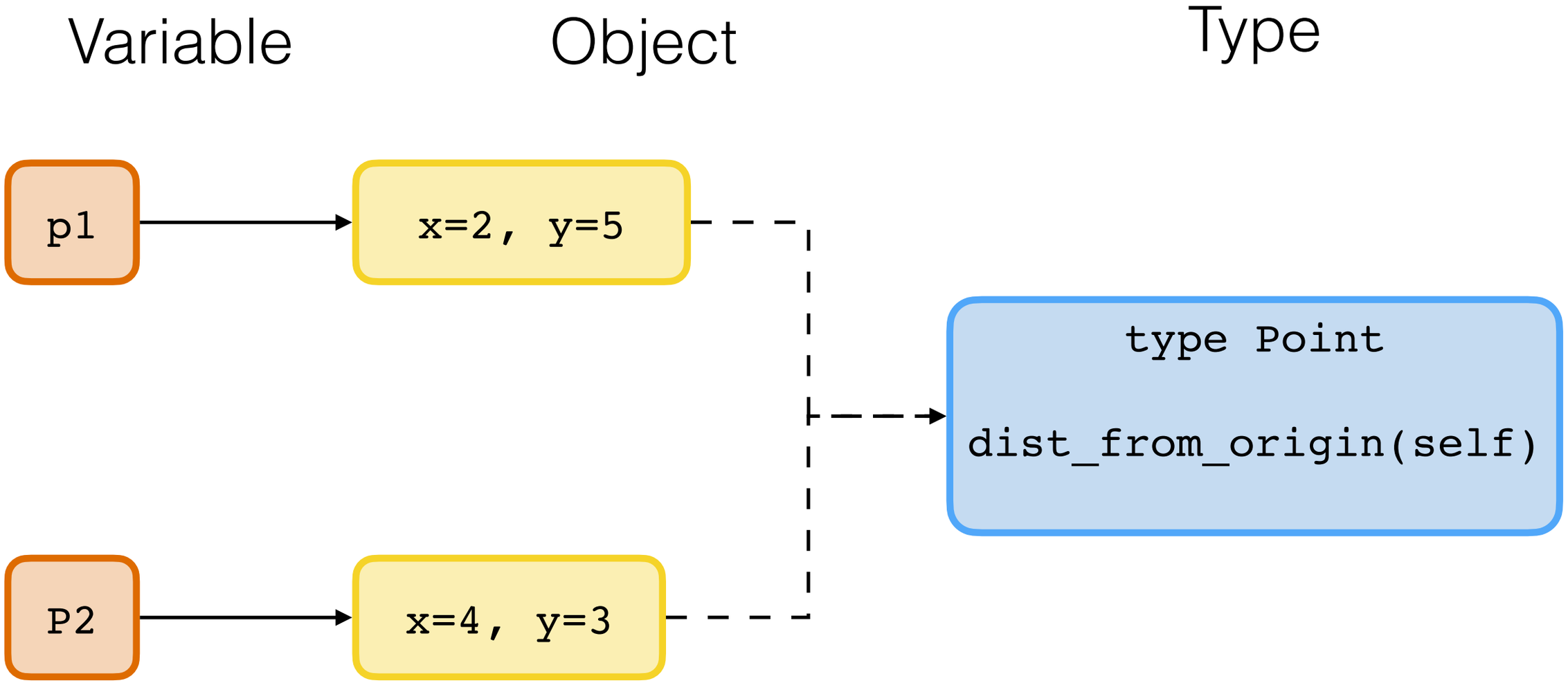
A `Point` class



Distance from origin



A `Point` class



Distance to origin

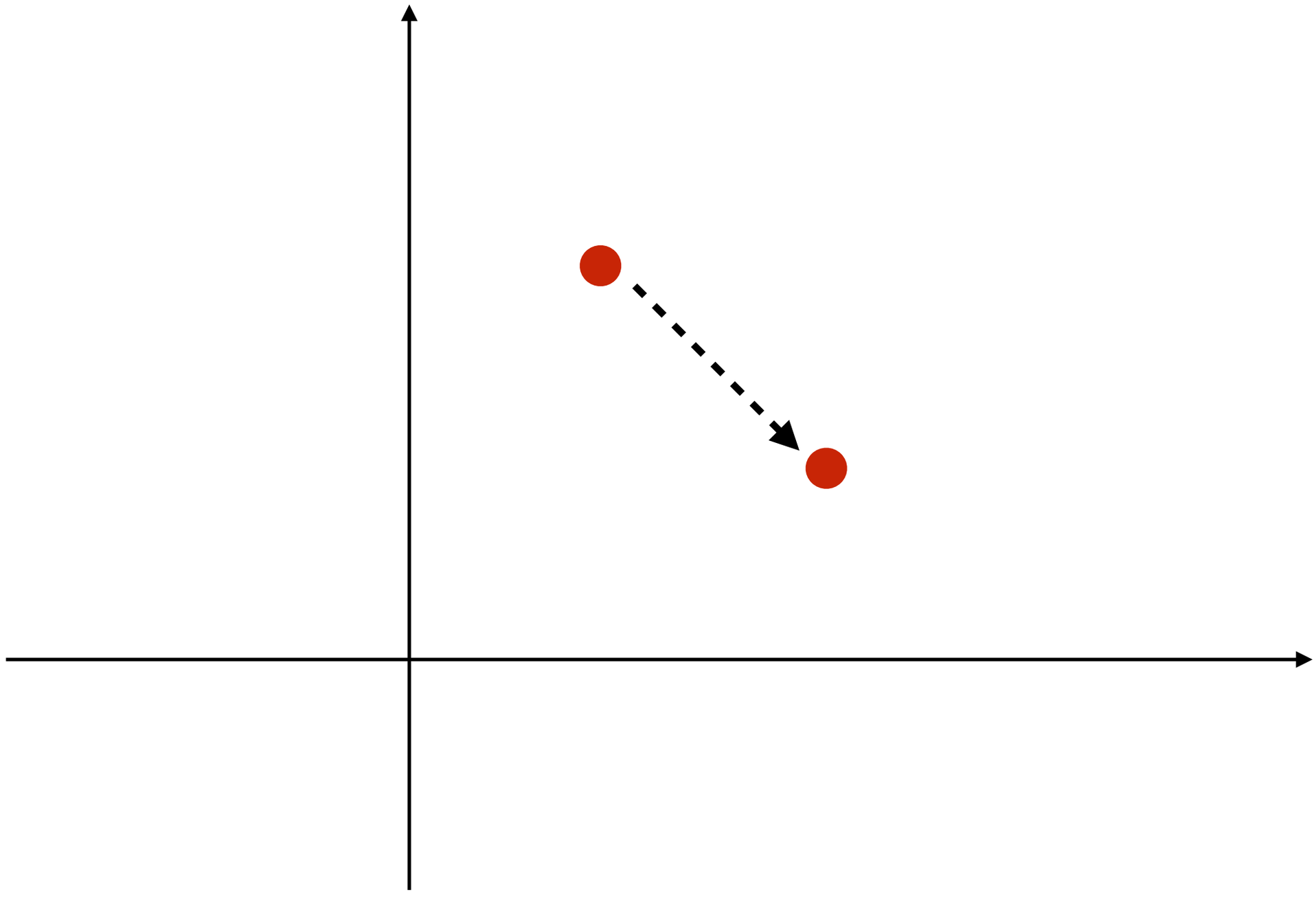
```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def dist_from_origin(self):
        return ...


p = Point(4, 3)
dist = p.dist_from_origin() # Point.dist_from_origin(p)
```

Solution

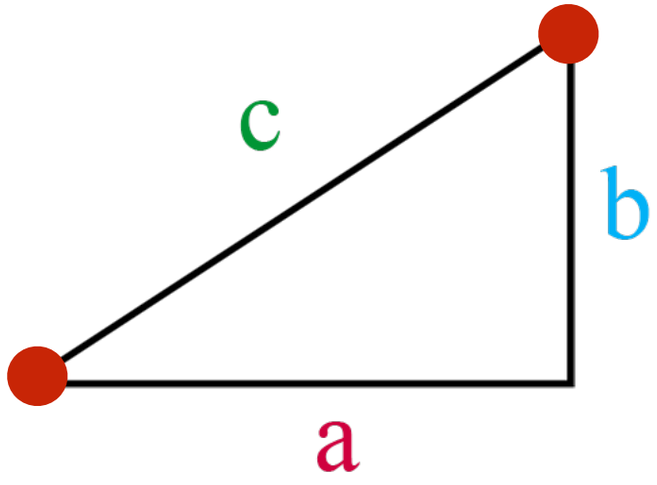
Distance to another point



Distance to another point

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance(self, other):
        ...
        ...
        return ...
```



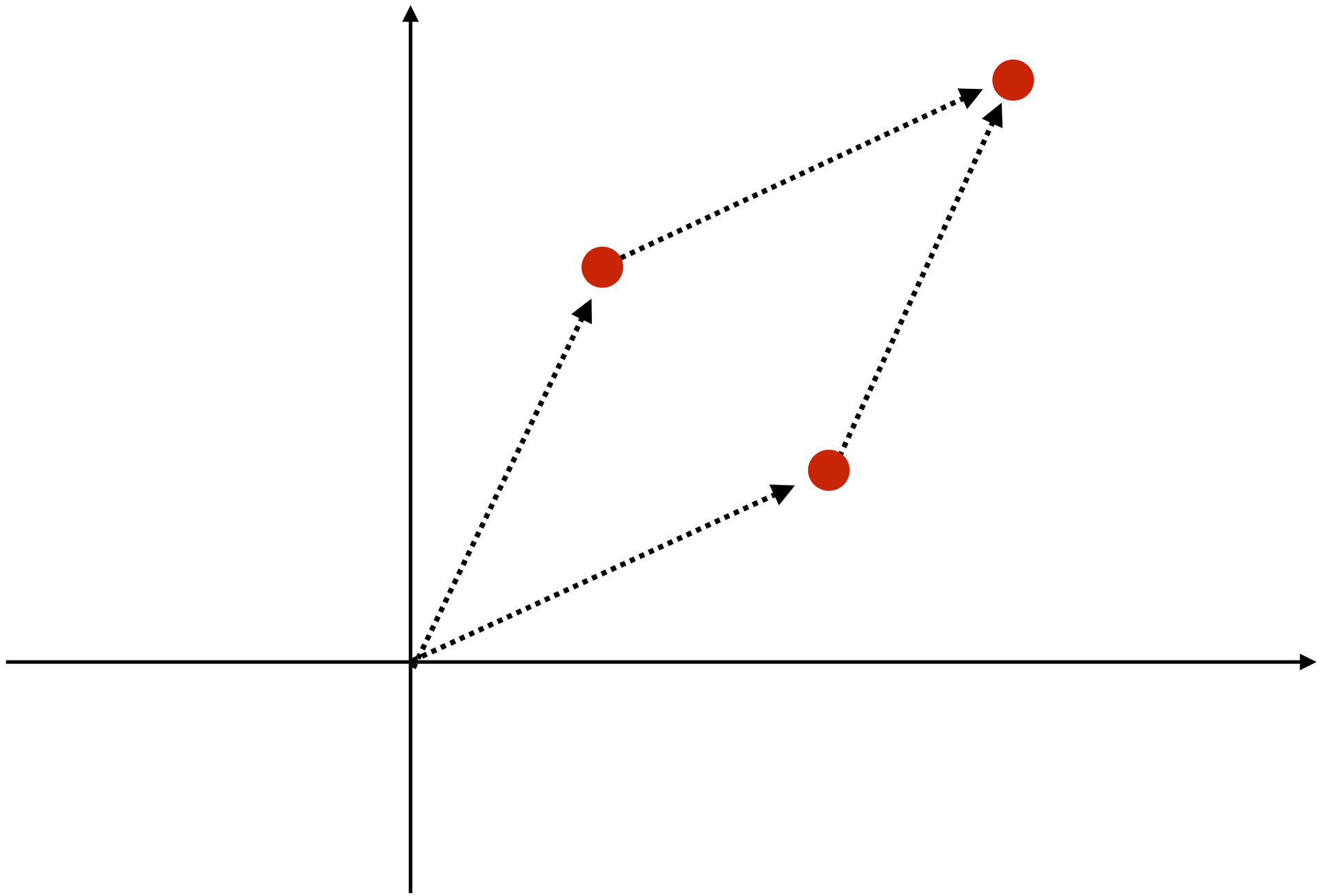
$$a^2 + b^2 = c^2$$

```
p1 = Point(4, 2)
p2 = Point(3, 7)
dist = p1.distance(p2)
```

```
c = (a**2 + b**2)**0.5
```

Solution

Adding points together



Adding points together

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        ...
        ...
        return ...

p1 = Point(4, 2)
p2 = Point(3, 7)
new_p = p1 + p2
```

Solution

Same position?

```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return ...

p1 = Point(4, 2)
p2 = Point(3, 7)
print(p1 == p2)
```

Solution

Codon usage in *Streptococcus* bacteria

Opgaven

Beregn frekvensen af codons, der koder for den samme aminosyre:

A:
- GCA: 0%
- GCC: 0%
- GCT: 100%
- GCG: 0%

C:
- TGC: 0%
- TGT: 100%

E:
- GAG: 33%
- GAA: 66%

...

Optælling

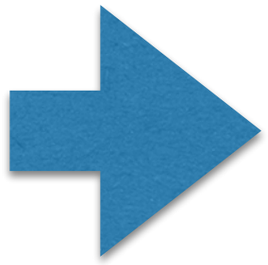
ORF



```
GCA:  0
GCC:  0
GCT:  7
GCG:  0
TGC:  0
TGT:  5
GAG:  4
GAA:  8
...
```

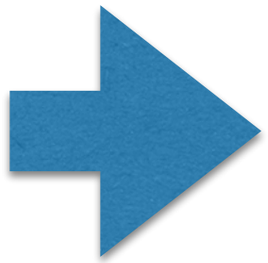
Gruppering

```
GCA: 0
GCC: 0
GCT: 7
GCG: 0
TGC: 0
TGT: 5
GAG: 4
GAA: 8
...
```



```
A:
    GCA: 0
    GCC: 0
    GCT: 7
    GCG: 0
C:
    TGC: 0
    TGT: 5
E:
    GAG: 4
    GAA: 8
...
```

Normalisering



```
A:
    GCA: 0
    GCC: 0
    GCT: 7
    GCG: 0
C:
    TGC: 0
    TGT: 5
E:
    GAG: 4
    GAA: 8
...
```

```
A:
    GCA: 0
    GCC: 0
    GCT: 1
    GCG: 0
C:
    TGC: 0
    TGT: 1
E:
    GAG: 0.33
    GAA: 0.66
...
```

Samlet set

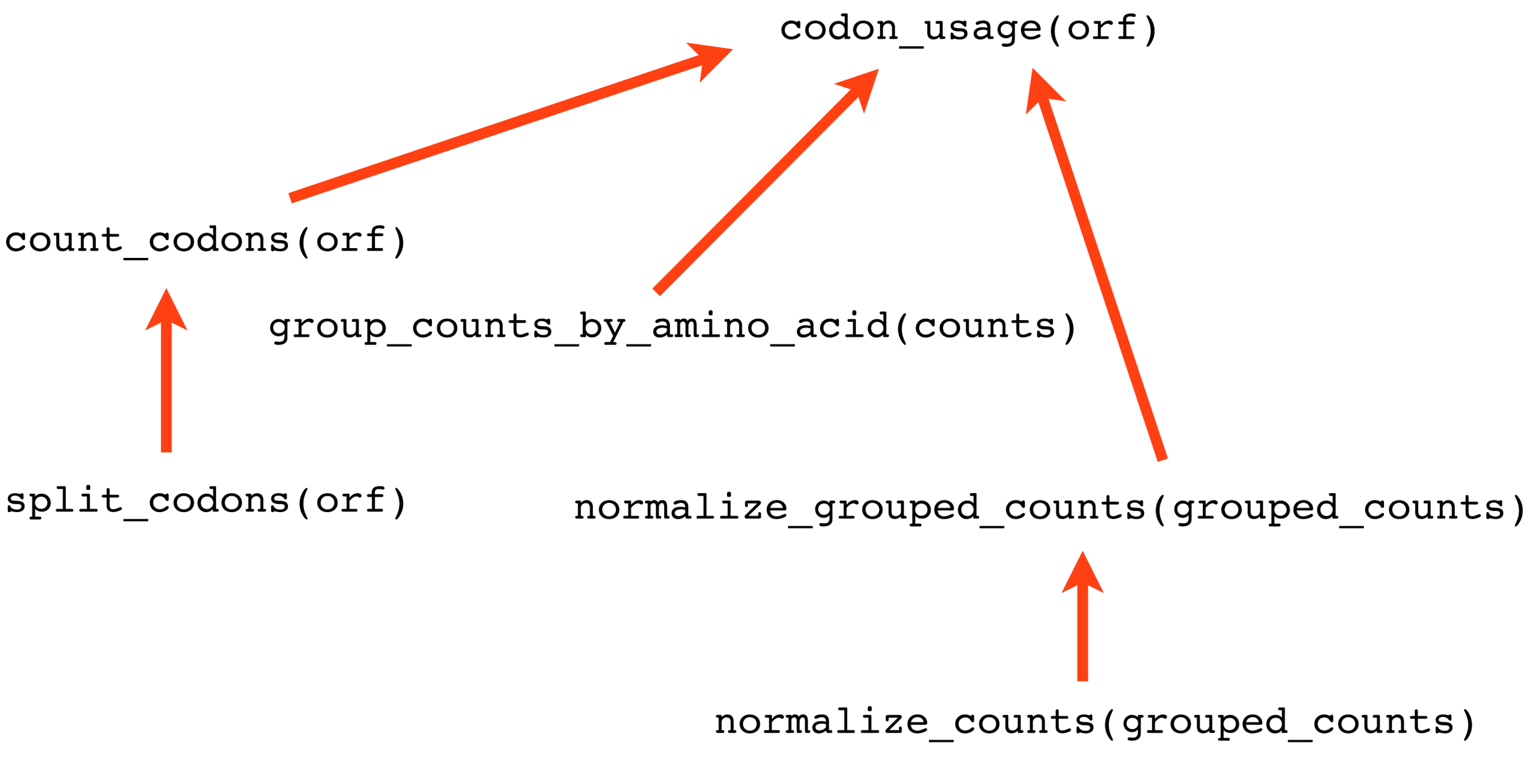
ORF

```
GCA: 0
GCC: 0
GCT: 7
GCG: 0
TGC: 0
TGT: 5
GAG: 4
GAA: 8
...
```

```
A:
    GCA: 0
    GCC: 0
    GCT: 7
    GCG: 0
C:
    TGC: 0
    TGT: 5
E:
    GAG: 4
    GAA: 8
...
```

```
A:
    GCA: 0.0
    GCC: 0.0
    GCT: 1.0
    GCG: 0.0
C:
    TGC: 0.0
    TGT: 1.0
E:
    GAG: 0.33
    GAA: 0.66
...
```

Outline


codon_usage(orf)
count_codons(orf)
group_counts_by_amino_acid(counts)
split_codons(orf)
normalize_grouped_counts(grouped_counts)
normalize_counts(grouped_counts)

```
counts = {'GCA': 0, 'GCC': 0, 'GCT': 7, 'GCG': 0,
          'TGC': 0, 'TGT': 5, 'GAG': 4, 'GAA': 8 }
```

```
grouped_counts = {'A': {'GCA': 0,
                        'GCC': 0,
                        'GCT': 7,
                        'GCG': 0 },
                  'C': {'TGC': 0,
                        'TGT': 5 },
                  'E': {'GAG': 4,
                        'GAA': 8 } }
```

```
counts = {'GCA': 0, 'GCC': 0, 'GCT': 7, 'GCG': 0,
          'TGC': 0, 'TGT': 5, 'GAG': 4, 'GAA': 8 }
```

```
grouped_counts = {}

grouped_counts['E'] = {}
grouped_counts['E']['GAA'] = counts['GAA']
grouped_counts['E']['GAG'] = counts['GAG']
```

```
grouped_counts = {'E': {'GAG': 4, 'GAA': 8 } }
```